

Gianni Forlastro

Introduzione a GIT

Esercizi



Introduzione a GIT - Esercizi

Esercizio 1

1. Installare GIT sul proprio computer
(<https://git-scm.com/downloads>)
2. Creare un account su GitHub utilizzando il proprio indirizzo di posta privato (consigliato)
3. Creare un account su GitLab utilizzando il proprio indirizzo di posta privato (consigliato)

Consiglio di utilizzare lo stesso indirizzo di posta per entrambi gli account, per avere un'unica mail di riferimento

Introduzione a GIT - Esercizi

Esercizio 2

1. Creare un repository su GitHub chiamato “TestJavaVOSTRONOME” (aggiungendo quindi il vostro nome al repository)
2. Creare un progetto Java, composto da un’unica classe chiamata “TestGitHubJava”. Nel main dovrà contenere una stampa a video di “Sono nel main di Test GitHub VOSTRO NOME”.
3. Salvare il progetto su GitHub, utilizzando il terminale.
4. Verificare su GitHub di aver salvato correttamente il progetto.

Introduzione a GIT - Esercizi

Esercizio 3

1. Creare un repository su GitLab chiamato “TestJavaVOSTRONOME” (aggiungendo quindi il vostro nome al repository)
2. Creare un progetto Java, composto da un’unica classe chiamata “TestGitLabJava”. Nel main dovrà contenere una stampa a video di “Sono nel main di Test GitLab VOSTRO NOME”.
3. Salvare il progetto su GitLab, utilizzando il terminale.
4. Verificare su GitLab di aver salvato correttamente il progetto.

Introduzione a GIT - Esercizi

Esercizio 4

1. Modificare il progetto “TestJava” di GitLab, aggiungendo la stampa a video “Stampo nuova riga su GITLab”
2. Salvare nuovamente il progetto su GITLab.
3. Verificare di aver modificato SOLO il progetto di GITLab e non quello di GitHub. Questo passaggio vi mostrerà come i due repository sono completamente separati.

Introduzione a GIT - Esercizi

Esercizio 5

1. Accordarsi con un compagno di corso e abilitare il progetto di GitHub anche alla sua mail (Settings > Manage Access)
2. La stessa cosa farà il vostro compagno di corso (quindi ognuno di voi sarà abilitato al progetto dell'altro)
3. Scaricare il progetto del vostro compagno su una nuova cartella del workspace locale
4. Modificare il main aggiungendo una stampa a video “Nuova stampa a video GitHub modificata da VOSTRO NOME”

Introduzione a GIT - Esercizi

Esercizio 5

5. Effettuare il commit+push del progetto nuovo
6. Effettuare il pull del progetto personale (originario) e verificarne le modifiche

Introduzione a GIT - Esercizi

Esercizio 6 (Opzionale)

1. Modificare a piacimento il progetto Java per creare conflitti e provare a gestirli e risolverli.
2. Creare un nuovo branch, con una funzionalità nuova (esempio una nuova funzione richiamata nel main) e provare a fare il merge.
3. Esercitarsi con i vari comandi visti a lezione.

Introduzione a GIT - Esercizi

Esercizio 7a

Creare una cartella "EsercizioGit" con al suo interno i file A.java, B.java.

```
class A {  
    private void itNeverChanges() {};  
}  
class B {  
    private void itWillChange() {};  
}
```

Con l'apposito comando, creare un nuovo repository a partire dalla cartella.
Verificare lo stato dei file: sono tracked o untracked?

Aggiungere i file al repository così che risultino tracked (Unmodified, Modified o Staged).
Verificare nuovamente lo stato dei file.

Introduzione a GIT - Esercizi

Esercizio 7b

Una volta aggiunti tutti i file, eseguire il primo commit inserendo come messaggio "Initial import"

Modificare B.java come segue:

```
class B {  
    private void itWillChange() {  
        // I'm a subtle change in the source code.  
    };  
}
```

Verificare nuovamente lo stato dei file, dovrete vedere che git ha rilevato la modifica.

Eseguire direttamente il comando commit inserendo come messaggio "Useless commit.": siccome non abbiamo aggiunto il file, Git ci comunica che il nuovo commit non contiene alcuna modifica. Utilizzare l'apposito comando per vedere quali file abbiamo modificato ma non inserito nell'area di Stage.

Aggiungere il file ed eseguire un commit con messaggio "I wrote the first comment.". Infine, visualizzare la storia dei commit fino a questo punto.

Introduzione a GIT - Esercizi

Esercizio 8

Compilare le due classi inserendo i binari in una nuova cartella "bin" e inserire un file "notes.txt" nella cartella principale. Aggiungere un file .gitignore tale che il nostro repository non sincronizzi i file .txt e la cartella bin.

Effettuare un commit del solo file .gitignore con il messaggio ".gitignore added" (ricordarsi di aggiungerlo ai file nell'area di stage).

A questo punto, il repository dovrebbe "ignorare" correttamente i file in questione. Ricordiamo che notes.txt, la cartella bin e il suo contenuto non erano presenti all'ultimo commit.

Con il comando `git add --all` aggiungiamo tutto ciò che non abbiamo aggiunto in precedenza. Verificare con il comando `git status` che non ci sia nulla di nuovo: se così non fosse, lanciate il comando `git reset` (rimuove tutti i file dall'area di stage) e rivedete il vostro .gitignore.

Introduzione a GIT - Esercizi

Esercizio 9a

Con l'apposito comando, verificate su quale branch state lavorando (dovrebbe essercene solo uno).

Create un nuovo branch chiamato "hotfix" e spostatevi su di esso. Create un nuovo file C.java, aggiungetelo e effettuate un commit con messaggio "First commit on the hotfix branch."

```
class C {  
    private void bornInTheTestingBranch() {}  
}
```

Tornate sul branch master ed eseguite il merge con il branch hotfix: per quale motivo non nascono conflitti?

Tornate sul branch master e modificate la classe B come segue.

```
class B {  
    private void itWillChange() {  
        // I'm a subtle change in the source code.  
        // I think it will work.  
    };  
}
```

Fate il commit della modifica con messaggio "Everything is fine."

Introduzione a GIT - Esercizi

Esercizio 9b

Create un nuovo branch "testing" e spostatevi su di esso. Modificate la classe B come di seguito.

```
class B {  
    private void itWillChange() {  
        // I'm a subtle change in the source code.  
        // Hi everyone, I'm the conflict.  
    };  
}
```

Fate il commit della modifica con messaggio "Git won't solve this."

Tornate sul branch master e provate a fare il merge con il branch testing: come mai genera un conflitto?

Risolvete il conflitto e controllate che sia tutto a posto.